Implementation of A* Algorithm

```
import heapq


def a_star(grid, start, goal):
    """Grid path-finding. 0 = free, 1 = wall. Heuristic: Manhattan distance."""
    h = lambda p: abs(p[0] - goal[0]) + abs(p[1] - goal[1])
    open_list = [(h(start), 0, start)]
    came_from, g_cost = {}, {start: 0}
    while open_list:
        _, g, cur = heapq.heappop(open_list)
        if cur == goal:
            path = [cur]
            while cur in came_from:
                cur = came_from[cur]
                path.append(cur)
            return path[::-1], g
        if g > g_cost[cur]:
            continue
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = cur[0] + dr, cur[1] + dc
            if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]) and grid[nr][nc] == 0:
                ng = g + 1
                if ng < g_cost.get((nr, nc), float("inf")):
                    g_cost[(nr, nc)] = ng
                    came_from[(nr, nc)] = cur
                    heapq.heappush(open_list, (ng + h((nr, nc)), ng, (nr, nc)))
    return None, float("inf")


def program4():
    print("\n=== 4. A* ALGORITHM ===")
    grid = [
        [0, 0, 0, 0, 0],
        [1, 1, 0, 1, 0],
        [0, 0, 0, 1, 0],
        [0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0],
    ]
    path, cost = a_star(grid, (0, 0), (4, 0))
    print("Path:", path)
    print("Cost:", cost)
    on_path = set(path)
    for r in range(5):
        print(" ".join("#" if grid[r][c] else ("*" if (r, c) in on_path else ".") for c in
            range(5)))


if __name__ == "__main__":
    program4()
```